

# 50 Rust Pattern Programs

Here are 50 different Rust programs to print patterns. Each pattern includes a description and the corresponding Rust code. Learn and explore different patterns with Rust.

## 1. Right Angle Triangle Pattern

```
fn main() {  
    let n: usize = 5;  
    for i in 1..=n {  
        println!("{}", " ".repeat(i));  
    }  
}
```

```
*  
* *  
* * *  
* * * *  
* * * * *
```

## 2. Inverted Right Angle Triangle Pattern

```
fn main() {  
    let n: usize = 5;  
    for i in (1..=n).rev() {  
        println!("{}", " ".repeat(i));  
    }  
}
```

```
* * * * *  
* * * *  
* * *  
* *  
*  
*
```

### 3. Pyramid Pattern

```
fn main() {  
    let n: usize = 5;  
    for i in 1..=n {  
        println!("{}", " ".repeat(n - i), "*" .repeat(i));  
    }  
}
```

```
  *  
 * *  
* * *  
* * * *  
* * * * *
```

### 4. Inverted Pyramid Pattern

```
fn main() {  
    let n: usize = 5;  
    for i in (1..=n).rev() {  
        println!("{}", " ".repeat(n - i), "*" .repeat(i));  
    }  
}
```

```
* * * * *  
 * * * *  
  * * *  
   * *  
    *
```

## 5. Diamond Pattern

```
fn main() {  
    let n: usize = 5;  
    for i in 1..=n {  
        println!("{}", " ".repeat(n - i), "*" .repeat(i));  
    }  
    for i in (1..n).rev() {  
        println!("{}", " ".repeat(n - i), "*" .repeat(i));  
    }  
}
```

```
  *  
 * *  
* * *  
* * * *  
* * * * *  
* * * * *  
* * * *  
 * * *  
  * *  
   *
```

## 6. Hollow Square Pattern

```
fn main() {  
    let n: usize = 5;  
    for i in 0..n {  
        for j in 0..n {  
            if i == 0 || i == n - 1 || j == 0 || j == n - 1 { print!("{}", " "); }  
            else { print!("{}", " "); }  
        }  
        println();  
    }  
}
```

```
* * * * *  
*       *  
*       *  
*       *  
*       *  
* * * * *
```

## 7. Full Square Pattern

```
fn main() {  
    let n: usize = 5;  
    for _ in 0..n {  
        println!("{}", "*" .repeat(n));  
    }  
}
```

```
* * * * *  
* * * * *  
* * * * *  
* * * * *  
* * * * *
```

## 8. Right Angle Triangle (Number Pattern)

```
fn main() {  
    let n: usize = 5;  
    for i in 1..=n {  
        let row = (1..=i).map(|x| x.to_string()).collect::<Vec<_>>().join(" ");  
        println!("{}", row);  
    }  
}
```

```
1  
1 2  
1 2 3  
1 2 3 4  
1 2 3 4 5
```

## 9. Inverted Right Angle Triangle (Number Pattern)

```
fn main() {  
    let n: usize = 5;  
    for i in (1..=n).rev() {  
        let row = (1..=i).map(|x| x.to_string()).collect::<Vec<_>>().join(" ");  
        println!("{}", row);  
    }  
}
```

```
1 2 3 4 5  
1 2 3 4  
1 2 3  
1 2  
1
```

## 10. Floyd's Triangle

```
fn main() {  
    let n: usize = 5;  
    let mut num = 1;  
    for i in 1..=n {  
        for _ in 0..i {  
            print!("{}", num);  
            num += 1;  
        }  
        println();  
    }  
}
```

```
1  
2 3  
4 5 6  
7 8 9 10  
11 12 13 14 15
```

## 11. Hollow Right Angle Triangle

```
fn main() {  
    let n: usize = 5;  
    for i in 1..=n {  
        for j in 1..=i {  
            if j == 1 || j == i || i == n { print!("{}", " "); }  
            else { print!("{}", " "); }  
        }  
        println();  
    }  
}
```

```
*  
* *  
*  *  
*   *  
* * * * *
```

## 12. Hollow Pyramid Pattern

```
fn main() {  
    let n: usize = 5;  
    for i in 1..=n {  
        print!("{}", " ", " ".repeat(n - i));  
        for j in 1..=(2 * i - 1) {  
            if j == 1 || j == 2 * i - 1 || i == n { print!("{}", " "); }  
            else { print!("{}", " "); }  
        }  
        println();  
    }  
}
```

```
    *  
  * *  
 *   *  
*     *  
*****
```

## 13. Hollow Diamond Pattern

```
fn main() {  
    let n: usize = 5;  
    for i in 1..=n {  
        println!("{}", " ".repeat(n-i), "", if i>1 { format!("{}", " ".repeat(2*i-3)) } else { String::new() });  
    }  
    for i in (1..n).rev() {  
        println!("{}", " ".repeat(n-i), "", if i>1 { format!("{}", " ".repeat(2*i-3)) } else { String::new() });  
    }  
}
```

```
  *  
 * *  
* * *  
 * * *  
  * *  
   *  
  * *  
 * * *  
* * *  
 * * *  
  * *  
   *
```

## 14. Hollow Diamond (Number Pattern)

```
fn main() {  
    let n: usize = 5;  
    for i in 1..=n {  
        println!("{}", " ".repeat(n-i), i);  
        if i > 1 { println!("{}", " ".repeat(2*i-3), i); }  
        println();  
    }  
    for i in (1..n).rev() {  
        println!("{}", " ".repeat(n-i), i);  
        if i > 1 { println!("{}", " ".repeat(2*i-3), i); }  
        println();  
    }  
}
```

```
  1  
 2 2  
3  3  
4    4  
5      5  
4    4  
3  3  
 2 2  
  1
```

## 15. Butterfly Pattern

```
fn main() {  
    let n: usize = 5;  
    for i in 1..=n {  
        println!("{}", i, " ".repeat(2*(n-i)), "**".repeat(i));  
    }  
    for i in (1..n).rev() {  
        println!("{}", i, " ".repeat(2*(n-i)), "**".repeat(i));  
    }  
}
```

```
*      *  
**     **  
***    ***  
****   ****  
*****  
*****  
****   ****  
***    ***  
**     **  
*      *
```

## 16. Hollow Number Pyramid

```
fn main() {  
    let n: usize = 5;  
    for i in 1..=n {  
        println!("{}", i, " ".repeat(n - i));  
        for j in 1..=i {  
            if j == 1 || j == i || i == n { print!("{}", j); }  
            else { print(" "); }  
        }  
        println();  
    }  
}
```

```
1  
1 2  
1 2 3  
1 2 3 4  
1 2 3 4 5
```



## 17. Full Star Pyramid

```
fn main() {  
    let n: usize = 5;  
    for i in 1..=n {  
        println!("{}", " ".repeat(n-i), "** ".repeat(i));  
    }  
}
```

```
  *  
 * *  
* * *  
* * * *  
* * * * *
```

## 18. Inverted Full Star Pyramid

```
fn main() {  
    let n: usize = 5;  
    for i in (1..=n).rev() {  
        println!("{}", " ".repeat(n-i), "** ".repeat(i));  
    }  
}
```

```
* * * * *  
* * * *  
* * *  
* *  
*
```

## 19. Left Aligned Pyramid Pattern

```
fn main() {  
    let n: usize = 5;  
    for i in 1..=n {  
        println!("{}", " ".repeat(i));  
    }  
}
```

```
*  
* *  
* * *  
* * * *  
* * * * *
```

## 20. Right Aligned Pyramid Pattern

```
fn main() {  
    let n: usize = 5;  
    for i in 1..=n {  
        println!("{}", " ".repeat(n-i), " ".repeat(i));  
    }  
}
```

```
      *  
     * *  
    * * *  
   * * * *  
  * * * * *  
 * * * * *
```

## 21. Pascal's Triangle

```
fn comb(n: u32, r: u32) -> u32 {
  (1..=r).fold(1, |acc, i| acc * (n - i + 1) / i)
}
fn main() {
  let n: u32 = 5;
  for i in 0..n {
    print!("{}", " ".repeat((n - i - 1) as usize));
    for j in 0..=i { print!("{}", comb(i, j)); }
    println();
  }
}
```

```
  1
 1 1
1 2 1
1 3 3 1
1 4 6 4 1
```

## 22. Zig-Zag Pattern

```
fn main() {
  let n: usize = 5;
  for i in 1..=n {
    if i % 2 == 0 { println!("{}", " "); }
    else { println!("{}", i); }
  }
}
```

```
*
*
*
*
*
```

## 23. Hourglass Pattern

```
fn main() {
    let n: usize = 5;
    for i in (1..=n).rev() { println!("{}", i, " ".repeat(n-i), "*" .repeat(i)); }
    for i in 2..=n { println!("{}", i, " ".repeat(n-i), "*" .repeat(i)); }
}
```

```

  * * * * *
 * * * *
  * * *
   * *
    *
   * *
  * * *
 * * *
* * * * *
```

## 24. Diamond Shape with Numbers

```
fn main() {
    let n: usize = 5;
    for i in 1..=n { println!("{}", " ".repeat(n-i), format!("{}", i).repeat(i)); }
    for i in (1..n).rev() { println!("{}", " ".repeat(n-i), format!("{}", i).repeat(i)); }
}
```

1  
2 2  
3 3 3  
4 4 4 4  
5 5 5 5 5  
4 4 4 4  
3 3 3  
2 2  
1

## 25. Hollow Rhombus Pattern

```
fn main() {  
    let n: usize = 5;  
    for i in 0..n {  
        print!("{}", " ".repeat(i));  
        for j in 0..n {  
            if i == 0 || i == n - 1 || j == 0 || j == n - 1 { print!("{}", " "); }  
            else { print(" "); }  
        }  
        println();  
    }  
}
```

```
* * * * *  
*       *  
  *     *  
   *   *  
    * *  
     * * * * *
```

## 26. Numeric Pyramid

```
fn main() {  
    let n: usize = 5;  
    for i in 1..=n {  
        let row = (1..=i).map(|x| x.to_string()).collect::<Vec<_>>().join(" ");  
        println!("{}", row, " ".repeat(n-i), row);  
    }  
}
```

```
      1  
     1 2  
    1 2 3  
   1 2 3 4  
  1 2 3 4 5
```

## 27. Hollow Diamond with Numbers

```
fn main() {  
    let n: usize = 5;  
    for i in 1..=n {  
        print!("{}", " ".repeat(n-i), i);  
        if i > 1 { print!("{}", " ".repeat(2*i-3), i); }  
        println();  
    }  
    for i in (1..n).rev() {  
        print!("{}", " ".repeat(n-i), i);  
        if i > 1 { print!("{}", " ".repeat(2*i-3), i); }  
        println();  
    }  
}
```

```
  1  
 2 2  
3   3  
4   4  
5     5  
4     4  
3     3  
2     2  
  1
```

## 28. Reverse Pyramid with Numbers

```
fn main() {  
    let n: usize = 5;  
    for i in (1..=n).rev() {  
        let row = (1..=i).map(|x| x.to_string()).collect::<Vec<_>>().join(" ");  
        println!("{}", " ".repeat(n-i), row);  
    }  
}
```

```
1 2 3 4 5  
 1 2 3 4  
  1 2 3  
   1 2  
    1
```

## 29. Diamond Star Pattern

```
fn main() {  
    let n: usize = 5;  
    for i in 1..=n { println!("{}", " ".repeat(n-i), "*" .repeat(i)); }  
    for i in (1..n).rev() { println!("{}", " ".repeat(n-i), "*" .repeat(i)); }  
}
```

```
  *  
 * *  
* * *  
* * * *  
* * * * *  
* * * * *  
* * * *  
 * * *  
  * *  
   *
```

## 30. Full Number Pyramid

```
fn main() {  
    let n: usize = 5;  
    for i in 1..=n {  
        let row = (1..=i).map(|x| x.to_string()).collect::<Vec<_>>().join(" ");  
        println!("{}", " ".repeat(n-i), row);  
    }  
}
```

```
  1  
 1 2  
1 2 3  
1 2 3 4  
1 2 3 4 5
```

## 31. Checkerboard Pattern

```
fn main() {  
    let n: usize = 5;  
    for i in 0..n {  
        for j in 0..n {  
            if (i + j) % 2 == 0 { print!("{}", " "); }  
            else { print!("{}", " "); }  
        }  
        println();  
    }  
}
```

```
* * *  
* * *  
* * *  
* * *  
* * *
```

## 32. Hollow Circle Pattern

```
fn main() {  
    let r: i32 = 3;  
    for y in -r..=r {  
        for x in -r..=r {  
            let d = x*x + y*y;  
            if d >= r*r - r && d <= r*r + r { print!("{}", " "); }  
            else { print!("{}", " "); }  
        }  
        println();  
    }  
}
```

```
***  
* *  
* *  
* *  
* *  
* *  
***
```



## 33. Triangle of Numbers

```
fn main() {  
    let n: usize = 5;  
    for i in 1..=n {  
        for j in 1..=i { print!("{}", j); }  
        println();  
    }  
}
```

```
1  
1 2  
1 2 3  
1 2 3 4  
1 2 3 4 5
```

## 34. Rhombus of Numbers

```
fn main() {  
    let n: usize = 5;  
    for i in 1..=n {  
        let row = (1..=i).map(|x| x.to_string()).collect::<Vec<_>>().join(" ");  
        println!("{}", row, " ".repeat(n-i), row);  
    }  
    for i in (1..n).rev() {  
        let row = (1..=i).map(|x| x.to_string()).collect::<Vec<_>>().join(" ");  
        println!("{}", row, " ".repeat(n-i), row);  
    }  
}
```

```
    1  
  1 2  
1 2 3  
1 2 3 4  
1 2 3 4 5  
1 2 3 4  
  1 2 3  
    1 2  
      1
```

## 35. Number Pattern (Decreasing)

```
fn main() {  
    let n: usize = 5;  
    for i in (1..=n).rev() {  
        let row = (1..=i).map(|x| x.to_string()).collect::<Vec<_>>().join(" ");  
        println!("{}", " ".repeat(n-i), row);  
    }  
}
```

```
1 2 3 4 5  
1 2 3 4  
1 2 3  
1 2  
1
```

## 36. Hollow Inverted Pyramid

```
fn main() {  
    let n: usize = 5;  
    for i in (1..=n).rev() {  
        println!("{}", " ".repeat(n - i));  
        for j in 1..=(2 * i - 1) {  
            if j == 1 || j == 2*i - 1 || i == n { println!("{}", " "); }  
            else { println(""); }  
        }  
        println();  
    }  
}
```

```
*****  
 *   *  
*   *  
 *   *  
  *   *  
   *
```

## 37. Cross Pattern

```
fn main() {  
    let n: usize = 5;  
    for i in 0..n {  
        for j in 0..n {  
            if i == j || i + j == n - 1 { print!("{}", " "); }  
            else { print!("{}", " "); }  
        }  
        println();  
    }  
}
```

```
*   *  
 *  *  
  *  
 *  *  
*   *
```

## 38. Number Inverted Pyramid

```
fn main() {  
    let n: usize = 5;  
    for i in (1..=n).rev() {  
        let row = (1..=i).map(|x| x.to_string()).collect::<Vec<_>>().join(" ");  
        println!("{}", row, " ".repeat(n-i), row);  
    }  
}
```

```
1 2 3 4 5  
 1 2 3 4  
  1 2 3  
   1 2  
    1
```

## 39. Right Angle Star Triangle

```
fn main() {  
    let n: usize = 5;  
    for i in 1..=n {  
        println!("{}", " ".repeat(n-i), "*" .repeat(i));  
    }  
}
```

```
      *  
     **  
    ***  
   ****  
  *****  
 *****
```

## 40. Left Angle Number Triangle

```
fn main() {  
    let n: usize = 5;  
    for i in 1..=n {  
        for j in 1..=i { print!("{}", j); }  
        println();  
    }  
}
```

```
1  
1 2  
1 2 3  
1 2 3 4  
1 2 3 4 5
```

## 41. Star Hollow Triangle

```
fn main() {  
  let n: usize = 5;  
  for i in 1..=n {  
    for j in 1..=i {  
      if j == 1 || j == i || i == n { print!("{}", " "); }  
      else { print!("{}", " "); }  
    }  
    println();  
  }  
}
```

```
*  
* *  
*  *  
*   *  
* * * * *
```

## 42. Upside Down Right Triangle

```
fn main() {  
  let n: usize = 5;  
  for i in (1..=n).rev() {  
    println!("{}", " ".repeat(i));  
  }  
}
```

```
* * * * *  
* * * *  
* * * *  
* * *  
* *  
*
```

## 43. Parallelogram Star Pattern

```
fn main() {  
    let n: usize = 5;  
    for i in 0..n {  
        println!("{}", " ".repeat(i), "*" .repeat(n));  
    }  
}
```

```
* * * * *  
 * * * * *  
  * * * * *  
   * * * * *  
    * * * * *
```

## 44. Inverted Parallelogram Pattern

```
fn main() {  
    let n: usize = 5;  
    for i in (0..n).rev() {  
        println!("{}", " ".repeat(i), "*" .repeat(n));  
    }  
}
```

```
    * * * * *  
   * * * * *  
  * * * * *  
 * * * * *  
* * * * *
```

## 45. Reverse Hollow Diamond

```
fn main() {  
    let n: usize = 5;  
    for i in (1..n).rev() {  
        println!("{}", " ".repeat(n-i), "", if i>1 { format!("{}", " ".repeat(2*i-3)) } else { String::new() });  
    }  
    for i in 2..n {  
        println!("{}", " ".repeat(n-i), "", format!("{}", " ".repeat(2*i-3)));  
    }  
}
```

```
* *  
 * *  
  * *  
   *  
  * *  
 * *  
* *  
 * *  
  * *  
   *  
  * *  
 * *  
  * *  
 * *  
  * *  
   *
```

## 46. Inverted Hollow Pyramid with Stars

```
fn main() {  
    let n: usize = 5;  
    for i in (1..n).rev() {  
        println!("{}", " ".repeat(n-i));  
        for j in 1..(2*i-1) {  
            if j == 1 || j == 2*i-1 || i == n { print!("{}", " "); }  
            else { print!("{}", " "); }  
        }  
        println();  
    }  
}
```

```
*****  
 *   *  
 * *  
 * *  
 *  
 *  
 *
```

## 47. Full Hollow Diamond

```
fn main() {  
    let n: usize = 5;  
    for i in 1..=n {  
        print!("{}", " ".repeat(n-i));  
        if i > 1 { print!("{}", " ".repeat(2*i-3)); }  
        println();  
    }  
    for i in (1..n).rev() {  
        print!("{}", " ".repeat(n-i));  
        if i > 1 { print!("{}", " ".repeat(2*i-3)); }  
        println();  
    }  
}
```

```
  *  
 * *  
* * *  
 * * *  
  * *  
   *  
  * *  
 * * *  
* * *  
 * * *  
  * *  
   *
```

## 48. Rectangle of Numbers

```
fn main() {  
    let rows: usize = 3;  
    let cols: usize = 5;  
    for _ in 0..rows {  
        for j in 1..=cols { print!("{}", j); }  
        println();  
    }  
}
```

```
1 2 3 4 5  
1 2 3 4 5  
1 2 3 4 5
```



## 49. Reverse Star Pattern

```
fn main() {  
    let n: usize = 5;  
    for i in (1..=n).rev() {  
        println!("{}", "*".repeat(i));  
    }  
}
```

```
*****  
****  
***  
**  
*
```

## 50. Pyramid of Numbers

```
fn main() {  
    let n: usize = 5;  
    for i in 1..=n {  
        let row = (1..=i).map(|x| x.to_string()).collect::<Vec<_>>().join(" ");  
        println!("{}", row);  
    }  
}
```

```
1  
1 2  
1 2 3  
1 2 3 4  
1 2 3 4 5
```